UNIVERSITY OF MILANO-BICOCCA

Department of Computer Science, Systems and Communication

Master's Degree Programme in **Computer Science**

Innovative Management of Service Requests: From a Distributed Monolith to a Microservices Architecture, with RAG-Enhanced Reccomendation System

Advisor: Prof. Leonardo Mariani

Master's Thesis by:
Nicolas Guarini
Student ID: 918670

Academic Year 2024–2025

"What I cannot create, I do not understand"

- Richard Feynman

Contents

Introduction

The technological evolution of recent decades has profoundly transformed the business landscape, making IT solutions a crucial factor for the success of enterprises. In this context, Elmec Informatica S.p.A. stands out as a significant player in the Information Technology sector in Italy. Founded in 1971 and headquartered in Varese, Elmec combines extensive industry experience with a strong focus on innovation, offering services and solutions that cover a wide range of business needs, offering advanced IT solutions for medium and large enterprises.

The company manages four datacenters (including one Tier IV certified), provides Hybrid IT services, digital workspace and Device-as-a-Service solutions, and is a key partner for cybersecurity and regulatory compliance. With over 450 certified technicians, Elmec integrates cloud technologies (in collaboration with AWS and GAIA-X), promotes environmental sustainability, and stands out for its innovation through its Technological Campus and ongoing investments in professional training.

The primary context of this internship revolves around the redesign and enhancement of the existing *Service Request Portal (SRP)*, which is essential for managing client service requests efficiently.

A Service Requests is a formal request submitted by a customer to obtain specific services or actions within their IT infrastructure. These requests can encompass a wide range of activities, from creating new user accounts in the Active Directory system to managing user permissions, resetting passwords, and deploying software updates.

Various challenges have been identified within the current architecture, particularly concerning the dynamic forms used by customers for submitting the service requests, the management of customizable fields, and the integration with external data sources.

The main objectives of this internship are:

- **Analysis of the Existing System**: Conduct a comprehensive analysis of the current state of the SRP system, identifying weaknesses and inefficiencies that hinder its performance and user experience;
- **Architectural redesign**: Propose and implement a complete architectural redesign of the system;
- **Modular and Scalable Solution**: Design a modular and scalable solution that can accomodate various client-specific configurations;

- **Migrations**: Plan and execute a full migration of databases and other company services/platforms that use SRP.
- **Fine-tuned LLMs integration**: Explore opportunities for integrating specifically fine-tuned LLM systems into the SR workflow.

Chapter 1

Overview of the existing System

The internship project was not focused on creating an entirely new system from scratch, but rather on a deep redesign and restructuring of an existing system developed many years ago using legacy technologies and affected by several issues and critical limitations.

Before discussing the architectural, technological, and implementation choices that were made, it is necessary to carry out a thorough analysis of the current system, in order to understand its behavior, operational flows, technologies used, the main issues to be addressed, how to resolve them, how to prevent them from reoccurring in the future, and how to ensure that the new system is scalable and maintainable.

1.1 Service Requests Workflow

The lifecycle of a Service Request within the current system is orchestrated through a series of defined states and automated processes, involving interactions between various components and human operators. The flow is designed to manage SRs from initiation to completion, handling approvals, execution, and potential errors [14].

In Figure 1.1 [14], a simple and high-level flowchart is shown that illustrates the main flows and scenarios. Instead, in Figure 1.2 a complete and detailed flowchart is shown that illustrates all the details about every flow, including information on the jobs and components that manage the various sections of an SR's workflow [14].

1.1.1 SR Initiation and Initial State Management

A Service Request can be initiated through two primary channels [14]:

- **Via MyElmec Portal**: Clients can submit an SR by completing a dedicated form from the MyElmec portal. Upon saving the form, a ticket is created in the HelpdeskAdvanced (HDA) system with a `QUALIFIED` status, and a corresponding Service Request is generated in SRP with a `NEW` status. These two entities are then linked via the HDA Ticket ID;
- **Via HDA Ticket by Elmec Operator**: Operators have the ability to associate a catalog SR directly with a ticket they are managing within the

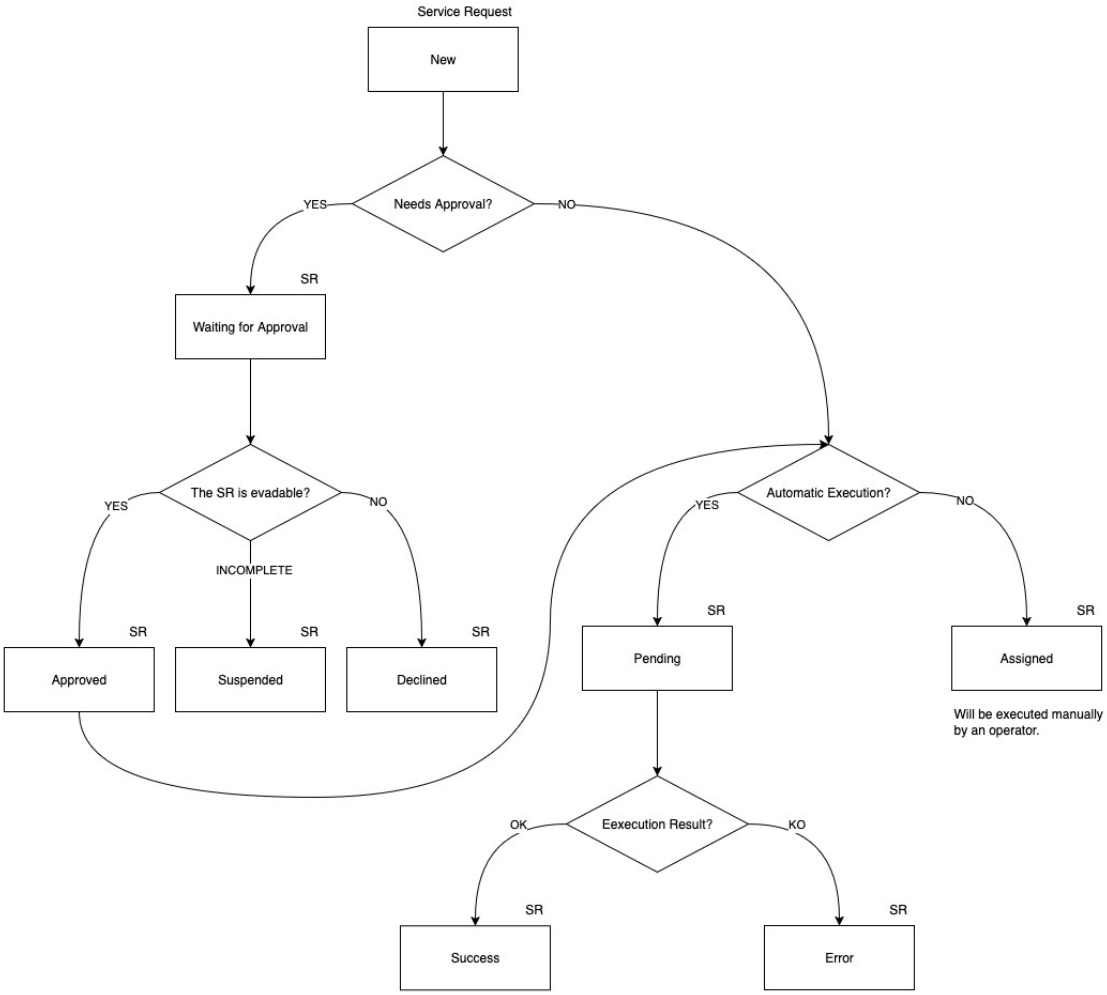

Figure 1.1: High-Level Service Request Lifecycle

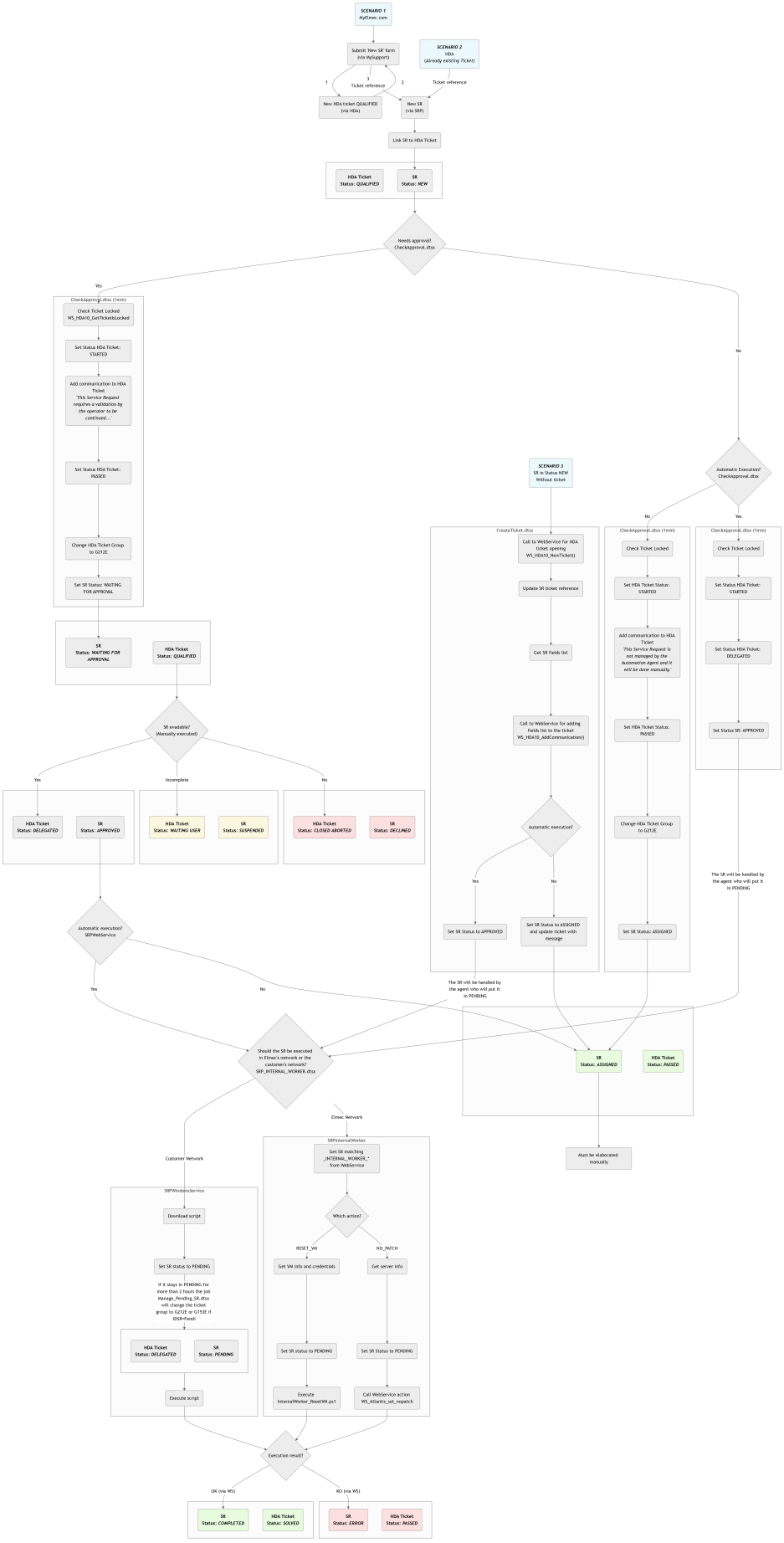

Figure 1.2: AS-IS Complete Service Request Execution Workflow

HDA interface. When the ticket is saved in a `DELEGATED` status, the SR is created in SRP with a `NEW` status, linked to the ticket, and immediately triggers the SR workflow within SRP.l

Following its creation, the system evaluates whether the Service Request requires approval by the operators. The logic for setting the initial SR and HDA ticket states is as follows [14]:

- **If approval is required**: the Service Request is set to `WAITING FOR APPROVAL` status, and the corresponding HDA ticket is set to `QUALIFIED`. A communication is added to the ticket, indicating that the SR needs validation from an operator before execution;
- **If no approval is required and it's linked to an automatism**: the Service Request enters a `PENDING` state, and the HDA ticket is set to `DELEGATED`;
- **If no approval is required and it's not linked to an automatism**: the Service Request is `ASSIGNED`, and the HDA ticket is set to `PASSED`. A communication is added to the ticket, noting that the SR requires manual processing by an operator.

1.1.2 Approval and Execution Workflows

For SRs requiring approval (`WAITING FOR APPROVAL` state with an HDA `QUALIFIED` ticket), an operator's intervention is necessary to validate the request. The operator has three possible actions [14]:

- **Cancel the SR**: the HDA ticket is set to `CLOSED ABORTED`, the SR status becomes `DECLINED`, and a notification email is sent to the requesting client;
- **Request client modification**: the HDA ticket is set to `WAITING USER`, the SR status becomes `SUSPENDED`, and a notification email is sent to the client. The workflow pauses until the client modifies the SR via Elmec.com;
- **Approve the SR**: the HDA ticket is set to `DELEGATED`, and the SR status becomes `APPROVED`.

Once a SR is `APPROVED`, SRP manages its subsequent flow based on whether it is linked to an automatism [14]:

- **If the SR is linked to an automatism**: the SR transitions to a `PENDING` state, while the HDA ticket remains `DELEGATED`. This implies it's awaiting automated execution;
- **If the SR is not linked to an automatism**: the SR is immediately set to `ASSIGNED`, and the HDA ticket is moved to `PASSED`. A note is added to the ticket indicating that the SR requires manual processing.

An SR in `PENDING` state is awaiting the execution of its associated automatism. The outcome of this automated execution determines the next state [21]:

- **Successful execution**: the SR is set to `COMPLETED`, and the corresponding HDA ticket is set to `SOLVED`;
- **Unsuccessful execution**: the SR transitions to an `ERROR` state, and the HDA ticket is set to `PASSED`. Communication regarding the error is added to the ticket for visibility within HDA.

The execution of Service Requests themselves can involve the SRP Agent on the client's Domain Controller for operations requiring client-side interaction (e.g., Active Directory or Exchange environment changes). The SRP Agent periodically checks `ELMEC-SRP01` for pending SRs. If found, it retrieves the SR details, loads the relevant PowerShell script, and executes it. The agent then informs ELMEC-SRP01 that the SR is running [21]. Monitoring is performed by a scheduled READLOG task on the client side, which checks the PowerShell script's log file every minute. A "KEY" in the log indicates completion, prompting the SRP Agent to inform ELMEC-SRP01 to set the SR to COMPLETED, triggering an HDA status update. An "ERROR" key indicates failure, leading to the log being sent to ELMEC-SRP01 and an error signal. A timeout mechanism also signals an error if completion is not detected within a specified duration [21].

1.2 Current System Architecture

The *SRP* system is fundamentally divided into two primary segments:

- Company Network;
- Client Network.

These segments interact and communicate exclusively through a *DMI Console* [16], which acts as a unidirectional bridge, transferring requests from the client network to the internal company network [16] [6].

The overall architecture is depicted in the diagram in figure 1.3 [16].

The system's operational flow involves several key components across both networks, orchestrating the processing of service requests.

1.2.1 Main Functionalities and Components

The current *SRP* system relies on a suite of interconnected components, each serving a specific role in the lifecycle of a service request.

Elmec's Network Components:

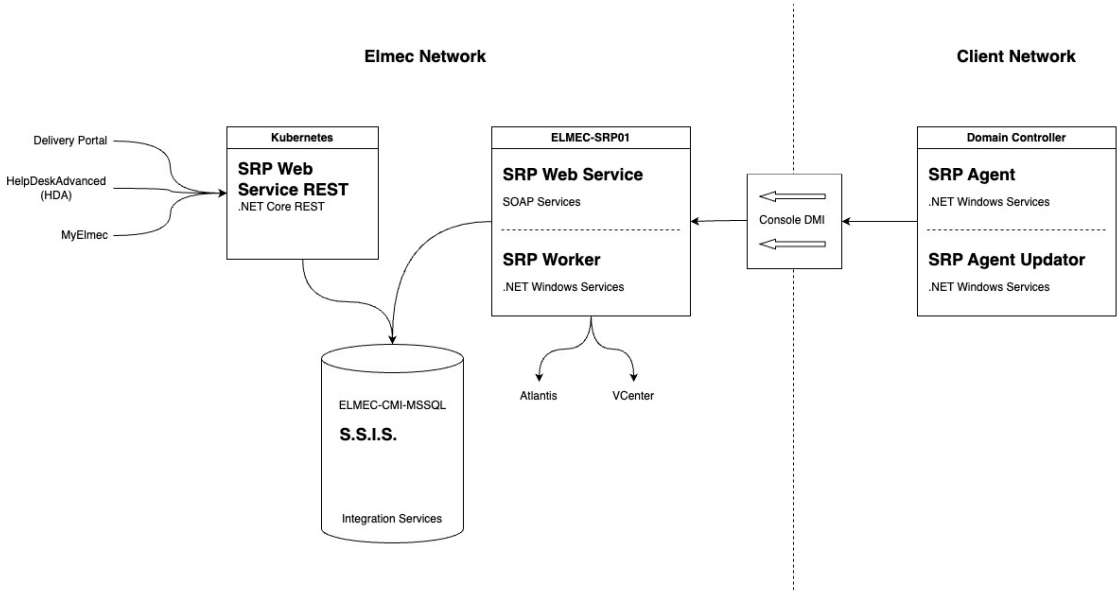

Figure 1.3: AS-IS System Architecture

- **SRP Web Sevice REST** [20]: This component, deployed on a Kubernetes cluster, exposes .NET Core RESTful services [16]. It serves as the primary interface for external systems such as:
  - **Delivery Portal**: An asset management system used to manage Service Request catalog configuration and Elmec's internal and client servers;
  - **HDA (HelpDeskAdvanced)**: A ticket management system. Each service request is intrinsically linked to an HDA ticket via an ID, and HDA is responsible for initiating and associating service requests with existing tickets.
  - **MyElmec**: A client-facing portal enabling customers to create, manage, and monitor the various services provided by Elmec (e.g., servers, management software, cloud solutions).

  The *SRP Web Service REST* communicates with the *ELMEC-CMI-MSSQL* server.
- **ELMEC-CMI-MSSQL**: This server hosts the central SQL Server Database for the SRP system. It is the persistent storage layer for all service request data and related information [16].
- **S.S.I.S. (SQL Server Integration Services)**: Running on the ELMEC-CMI-MSSQL server, SSIS packages are crucial for data transformation and import/export operations. They facilitate the transfer of data, such as master data (e.g., Active Directory users, Organizational Units, groups) sent by SRP Agents, into the database. Each SSIS package is designed to transform the received information for database insertion or vice versa [16].

- **ELMEC-SRP01**: This Windows Server functions as the core engine for service requests within the Elmec network. It houses all the PowerShell scripts associated with individual service requests, which are maintained by the Platform team [16]. Key components residing on ELMEC-SRP01 include:
  - **SRP WebServiceSoap**: This component provides SOAP services primarily for communication with the SRP Agent located on the client side. It interacts with the database to retrieve necessary information before exposing it to the agents for service request execution [19].
  - **SRP InternalWorker**: A Windows service that operates similarly to the SRP Agent but executes entirely within the Elmec network. This is utilized for service requests that do not require interaction with the client's network or Active Directory (e.g., "no patch" service requests). The InternalWorker queries the database every 10 seconds for "internal worker" type jobs (like removing a server from a patching session, or resetting a managed Virtual Machine), connects to the SOAP service, retrieves the Service Request ID and its fields, and initiates job execution. It also monitors logs every 30 seconds to determine the job's status [18].

The only component in the client's network is called **Domain Controller**, which represents the client's Active Directory domain controller, where the communication tools with Elmec are installed. It hosts:

- **SRP Agent (.NET Windows Service)**: Typically, one agent is configured per client domain, specifically for Active Directory (AD) and Exchange-related service requests. This agent performs two main functions [21]:
  - Every 8 hours, it reads master data (AD users, OUs, groups) from the domain controller and sends it to ELMEC-SRP01 for storage via SSIS.
  - Every 5 minutes, it queries ELMEC-SRP01 for pending Service Requests. Upon finding any, it requests detailed information, loads the corresponding PowerShell script, and executes it. After initiating the script, the agent informs ELMEC-SRP01 that the SR is "running" and proceeds to the next SR.
- **SRP Agent Updator (.NET Windows Service)**: This service continuously monitors the SRP Agent folder for a file with a .new extension. If detected, it stops the SRP Agent, updates it, and restarts it

1.2.2 Interactions and Communication between Customers and Company networks

The Communication between the company network and the clients' networks is a critical aspect of the SRP system's operations and is handled through the **DMI Console**.

The DMI console serves as a key infrastructural component, primarily acting as a communication bridge between the customer's network and Elmec's internal network. Specifically, for the SRP system, the DMI console is responsible for transferring service requests (SR) from the customer's network to Elmec's internal network, although it does not handle traffic in the opposite direction [6].

Operation of the DMI Console

The connectivity of the DMI appliances to Elmec is established through multiple OpenVPN connections, initiated by the appliance towards the Brunello 4 and Mendrisio Data Centers (for Swiss customers). The DMI consoles are not directly exposed to the Internet and are accessible only by authorized personnel. Hardware requirements for a DMI node include 4 GB of RAM, 4 cores, and 50 GB of disk space [6].

Following recent developments in the DMI, the new consoles use K3S for container management. K3S is a lightweight, certified Kubernetes distribution designed for edge, IoT, and CI/CD environments. It is characterized by being a single binary that reduces external dependencies and uses SQLite as the default database, greatly simplifying installation and management. It already includes essential components such as *containerd* [10].

All consoles are centrally managed via Rancher, and the deployment of the application stack is delegated to ArgoCD, which ensures that container versions are constantly updated. For security reasons, the console in the customer network by default exposes only SSH, while services such as SRP are selectively enabled and only when necessary. All application data present on the consoles is stored on a LUKS (Linux Unified Key Setup) encrypted volume[1], which can only be unlocked if the VPN tunnel is established. The operating system update is performed through a standard Elmec system, called PMD (Patching Management Dashboard) [6].

[1] A platform-independent hard disk encryption method that can be used to encrypt disks across a wide variety of tools. This not only ensures full compatibility and interoperability between different software but also guarantees that password management occurs in a secure and documented manner [4].

1.3 Problems and core issues of the current system

The analysis of the architecture and workflow of the current SRP system has revealed a number of critical issues that compromise its scalability, maintainability, and reliability. These problems are deeply rooted in the technological and design choices made years ago and are evident across several key areas of the system, from catalog management to script execution and the import of master data from Active Directory.

1.3.1 Architectural and Technological Challenges

The SRP system is characterized by a fragmented architectural structure and complex interdependencies between its main components, which leads to several critical issues.

Monolithic Frontend Component

The user interface, responsible for navigating the Service Request catalog and completing the associated forms, is an excessively large and complex Vue.js frontend component (nearly 11,000 lines). This architecture makes modifications, extensions, and client-specific customizations exceptionally challenging [12]. The logic for managing dynamic fields, their interdependencies, and the rules for compilation and validation are all handled at the frontend level through a long series of hard-coded conditional statements, such as the following:

```
<div v-if="field.desc == 'Domain'">
    ...
</div>
```

This, in addition to mixing business logic with presentation, represents a significant challenge for catalog configuration, as there is no centralized way to manage the fields and their relationships. Every change, therefore, requires a direct intervention on the code, making the system inflexible and difficult to maintain, especially in a context where the operator modifying the catalog is not a programmer and does not know how the frontend code is structured. For instance, if an operator wanted to add a new "Domain" field to a Service Request, this field would need to have the exact description "Domain" and type "domain"; otherwise, the Vue.js component would not render it correctly.

These dynamics make the system fragile and susceptible to frequent errors that waste time and resources for both clients and operators [12].

Overly Generic Backend REST Service

The only information about the fields returned by the backend REST service (`SRPWebSeviceREST`), besides the field name and its mandatoriness, is its *type*. It is therefore easy to imagine how this information, which should only be an indication of the component that needs to be rendered (e.g., textbox, select, radio button, etc.), can become an identifier for something much broader.

For example, a field of type `UPN` is a composite field derived from the value of another field, '`Display Name`', concatenated with an '`@`' symbol, and suffixed with the value from a select field whose options are the client's domains, provided by an external REST service (e.g., "`n.guarini@elmec.it`", where "`n.guarini`" is the value of the "`Display Name`" field, and "`elmec.it`" is the value of the "`Domain`" select field).

Despite this complexity, the REST service returns only a JSON document of this type:

```
{
    "idField": 1603,
    "descField": "UPN",
    "type": "upn",
    "mandatory": true
}
```

thus delegating to the frontend the responsibility of interpreting the type and rendering the field correctly.

Dependency on the SRP Agent

The import of data from clients' Active Directories and the actual execution of Service Requests are handled by an agent (`SRPWindowsService`) installed directly on the clients' Domain Controllers. This approach presents several challenges:

- **Security and Permissions**: The agent often requires credentials with elevated privileges (e.g., `domain-admin`), which clients are increasingly reluctant to grant.
- **Reliability and Control**: The configurations on the agent and the Domain Controller are complex and difficult to manage centrally, which can cause execution errors and data synchronization issues.
- **Scalability**: The need to install, configure, and maintain an agent for each client domain represents a significant operational effort. Alternatives such as more modern provisioning systems will be evaluated.
- **Obsolete Communication Patterns**: The unidirectional communication through the *DMI Console* creates a bottleneck and complicates

the management of asynchronous operations. Communication between components in the client's network and the company network is based on polling (e.g., the SRP Agent queries the server every 5 minutes), an approach that introduces latency and inefficiencies.

1.3.2 Code and Data Management Issues

Code and data persistence management is another critical aspect of the system, characterized by a lack of versioning and modern debugging tools.

Unversioned and Undebuggable Stored Procedures

The intensive use of Stored Procedures[2] in the MSSQL database represents one of the major weaknesses. These procedures contain fundamental business logic and are:

- **Unversioned**: There is no centralized version control system for Stored Procedures. Changes are made directly on the database, making it impossible to track changes, revert to previous versions, or test modifications in separate environments.
- **Difficult to Debug**: Debugging Stored Procedures is a complex and cumbersome process, often requiring direct access and specific SQL server tools, with the risk of affecting the production environment.

Inaccessible and Unversioned SSIS Jobs

SSIS packages[3] are essential for data import and export operations (e.g., AD master data). However, these jobs reside physically on the `ELMEC-CMI-MSSQL` server, and the only way to access and modify them is via a remote desktop. This approach prevents versioning and collaboration, limits accessibility by making maintenance and debugging an onerous and risky activity, and creates a *single-point-of-failure* and a strong coupling between the import/export business logic and the infrastructure.

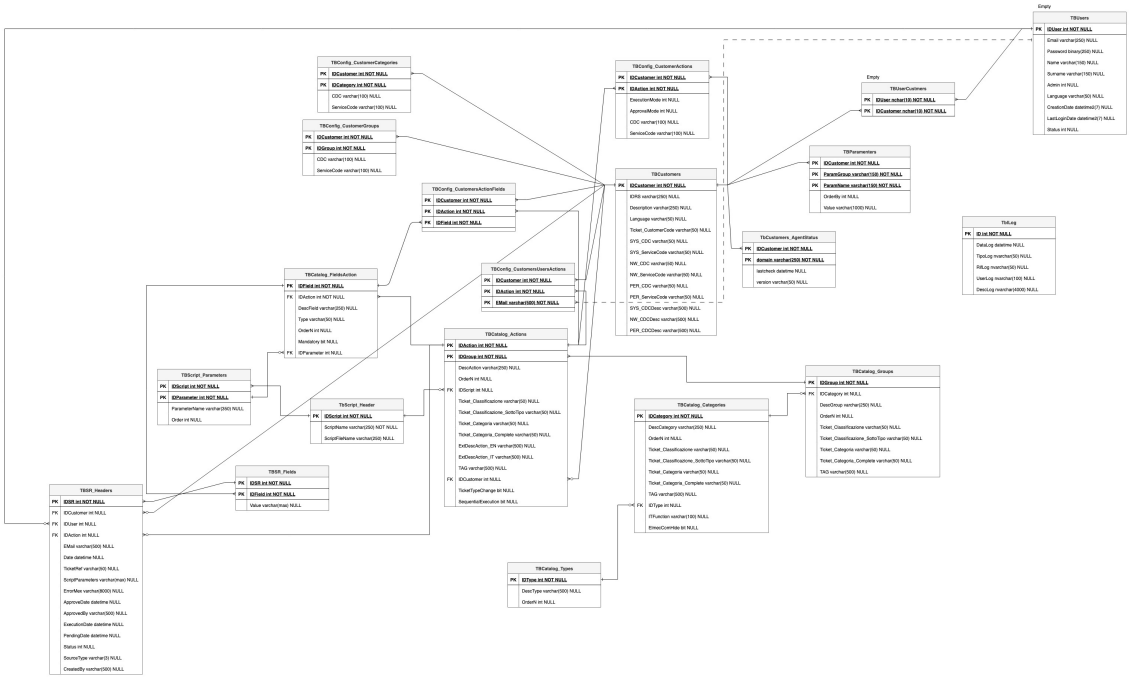

Figure 1.4: AS-IS Database ER Schema

1.3.3 Database Schema and Data Integrity Issues

The analysis of the database schema, represented in figure 1.4, has revealed a number of significant issues that affect the quality and integrity of the data managed by the SRP system.

Lack of Data Integrity and Consistency

One of the most significant problems concerns the lack of explicit referential integrity constraints. In the schema in figure 1.4, although the relationships between tables have been inferred manually, they are not defined at the database level through the use of foreign keys. This absence prevents the database from guaranteeing data integrity [3]: for example, a row in a child table could refer to a non-existent row in the parent table [25]. Such a scenario, known as a "dangling reference[4]", can generate anomalies and incon-

[2]Stored procedures are SQL code routines, or sets of instructions, that are pre-compiled and stored within a relational database. They act as logical units of execution, encapsulating business logic directly at the database level [22].

[3]SQL Server Integration Services (SSIS) is a platform for building data integration and workflow solutions. It is a key component of Microsoft SQL Server, primarily used for Extract, Transform, Load (ETL) operations, enabling the extraction of data from various sources, its transformation, and its loading into different destinations [17].

[4]The "dangling pointer" is a problem typically addressed in the context of low-level programming, where a pointer refers to a memory area that is no longer valid because it

sistencies in the system, making the data unreliable and debugging extremely complex [3].

Data Redundancy

Another issue is the high data redundancy and the large number of duplicate attributes across different tables [21]. This design, which deviates from the principles of normalization [2], leads to storing the same information in multiple locations, which not only increases the data volume but also introduces a high risk of inconsistencies [2].

Unclear and Inconsistent Naming Conventions

The naming convention for attributes and tables is inconsistent and unclear. The schema presents a mixture of conventions (e.g., `IDGroup` vs. `GroupID`, English names mixed with specific terms) and typos (e.g., `TBParamenters`), making it difficult to immediately understand the function of each field. In many cases, the naming is not explicit enough to describe the role of an attribute, requiring an in-depth knowledge of the system to correctly interpret the relationships and business logic. This lack of a unique and standardized vocabulary hinders collaboration among developers, complicates code maintenance, and increases the likelihood of errors.

1.3.4 Maintenance and Operational Challenges

According to information discussed in an internal meeting [15], the daily management of the system is made complex by several inefficiencies and obsolete design choices.

The current system design does not allow for the effective reuse of Service Requests and catalog configurations. Consequently, every modification or addition requires the creation of a new Service Request, even for small variations, leading to a innefficient and difficult-to-manage catalog, consisting of numerous duplicate SRs and fields. This, while functional, is not a scalable approach and leads to significant inefficiencies in catalog management itself. For example, if a Service Request needs to be modified for all clients who use it, the operator will have to modify not only the SR in question but also all the duplicate SRs for each client, with the risk of forgetting a modification or making mistakes. This approach, although probably conceived to prevent error propagation, makes the catalog difficult to navigate and maintain.

---

has been freed or because the pointer is used outside the context of the variable's existence to which it refers [8], but the same dynamic can also be applied in the context of databases, where the "pointers" are represented by foreign keys.

The new architecture will need to address these challenges by introducing a more robust, versioned system based on modern software design principles and patterns.

Chapter 2

Architectural Redesign

Given the numerous critical issues that emerged from the analysis of the current system, it is clear that a thorough redesign is necessary. This redesign must be able to perform the system's current functions more efficiently and in an optimized manner, while also satisfying a series of requirements that did not exist when the system was developed several years ago, and which the current system cannot manage because it was not designed to do so.

2.1 New System Requirements

The system can be divided into three main components: **Catalog Structure and Management**; **Service Request Execution**, and **Master Data Import**. The requirements for each of these areas will therefore be analyzed.

2.1.1 Catalog Structure and Management

This is the most substantial component of the three, dealing with everything concerning the definition, maintenance, and configuration of the actions executable by each customer. This therefore also includes the management of form fields, customer-specific customizations, scripts, their parameters, the target machine address retrieval logic, and so on.

Before proceeding with the analysis of the requirements, a clarification on the terminology to be used is necessary: an '*Action*' is understood as the formal definition of an operation that a client can execute on its infrastructure; a '*Service Request*' (SR), on the other hand, is understood as the effective execution of an Action. If a parallel with OOP is to be drawn, the Action can be seen as a class, and the Service Request as an object, an instance of the aforementioned class.

Actions

The Action entity can be seen as the core of the catalog component, and it is composed of the following main elements:

- General metadata, such as the name, an extended description, tags, a type, a group, a category, and other information related to administrative

and billing aspects;

- The fields of the corresponding form that must be filled by the client for its execution, including the relative logic tied to types, data validation, dependencies between various fields (e.g., fields that, once completed, "enable" other fields), autofill (e.g., fields whose values are autofilled with values from other fields), sensitive values, and so on;
- The script that will be executed in the customer's infrastructure to perform the effective required operation;
- Script parameters, which may be field values, but also additional parameters necessary for execution (e.g., AWS credentials, 365 credentials, etc. . .);
- The target machine on which the script must be executed.

Some of these dynamics are already implemented in the current system (with all the critical issues of the case, as already discussed in the previous chapter), while others are not. The crucial point, however, is that all these configuration aspects must be customizable for specific customers and for specific actions [15].

Given that the users of this system are very large clients (Fendi, Valentino, Lavazza, to name a few), it is easy to imagine that each of them has highly specific needs, which make direct reuse of Actions across multiple clients impossible. At the same time, however, the creation of infinite duplicated Actions that share almost everything, with only small differences (as happens in the current system [15]), must be avoided.

It is therefore necessary to implement a system that provides for the possibility of defining a sort of hierarchical structure, where a "parent" action is configured, which can be "inherited" by several child actions that will specify only the modifications, all without affecting the default action.

Fields

Together with actions, fields are central components on which a large part of the system is based. As already mentioned, fields are entities that can be extremely diverse, such as text fields, selects, multiple-choice selects, radio buttons, etc. In the current system, there are 86 different field types. This number is so high because, due to how the current system is built, the "type" information is the only way the backend communicates to the frontend what type of field is to be rendered [20]. In fact, there are many different types that share the same logic, perhaps simply using two different services to retrieve the data. The effective field typologies are therefore much fewer, and it is crucial that they are identified and generalized as much as possible.

The general idea is that the logic for fields must be moved to the backend,

which must return all the necessary information for the frontend to render the form: from field types, to validation regexes, to error messages, to any external services to be contacted to retrieve the options list, or to validate the data, and so on [15].

All this must obviously be compatible with the customization requirements specified in the previous section.

Regarding customizations, a typical example scenario could be the following: a customer uses an action, but for some reason requires two additional fields, which will then be needed by the script that will in turn require two additional parameters. Furthermore, for some fields, the client in question uses different validation rules and specific autofill templates. These customizations must be possible without modifying the default action and fields.

Regarding "particular" field types, the system must provide for [15]:

- Select fields whose options are customer-specific (e.g., "Domain" field, which is a select with all the customer's domains);
- Select fields whose options are global for the entire system (e.g., "Country" field, which provides a list of states that are the same for all customers);
- Select fields whose options are customer-specific and to obtain them an external service must be contacted, with the possible option to search (e.g., "Server" field, which is a select with all the customer's servers, returned by the Atlantis CMDB service);
- Autofilled text fields, meaning their values are obtained through a templating system (Jinja) that uses the values of other fields (e.g., "Logon Name" field, populated by concatenating the user's first and last name separated by, for example, a dot);
- Text fields whose validation occurs through regex (e.g., "Date" field, which must comply with the standard format);
- Text fields whose validation occurs through an external service (e.g., "Logon Name" field, which requires contacting the relevant Active Directory service for validation that a user with the same logon name does not already exist);
- Fields with sensitive values, whose values will therefore need to be saved encrypted, using some symmetric encryption algorithm (e.g., "Password" field);
- "Mixed" fields, composed of multiple "sub-fields" with different characteristics (e.g., "UPN" field, which is composed of the first letter of the first name, the last name, an at sign (`@`), and the value of a select whose options are obtained from an external API call);

It is important that the system is designed in a scalable way, so that if another

field typology were to be added in the future, it can be implemented with the minimum possible effort.

Configuration and Execution Consistency

A system for monitoring the consistency of the catalog configurations [15] will be necessary, in order to prevent the creation of invalid configurations, which would then lead to errors during SR execution.

For example, if a `select` type field is specified for an action, whose options are obtained from customer-specific parameters, but those parameters have not been defined for that customer, the system must report this inconsistency through some alerting system (e.g., email, ticket, monitoring dashboard). Alternatively, if an action is configured to be executed on a certain type of machine but the field necessary to select it is not among the action's fields, the system must report this inconsistency.

Similarly, a system for monitoring the consistency of SR executions must be implemented, in order to be able to report any errors that may have occurred during execution, such as an SR stuck in an "intermediate" state (e.g., `NEW`) for too long, or an SR that started execution (i.e., in `PENDING`) but never reached completion (i.e., in `COMPLETED` or `ERROR`) within a certain time limit (e.g., 2 hours).

Additional script parameters

Another topic that must be managed is that of script parameters. In the old system, there was granular control over every single parameter, which was linked to an action's script and possibly to a field in the corresponding Service Request. In practice, all the action's fields with their respective values were passed as parameters to each script, plus some additional parameters not linked to any field (e.g., Exchange Server, AD Sync Server, various credentials). This dynamic of additional parameters occurred through the definition of parameters with the foreign key on the field set to `NULL`, which were then intercepted by the stored procedure relating to the generation of the parameter string which, with hard-coded `if` statements (e.g., `IF @ParamName = 'ServerExc' BEGIN ...`), were valued accordingly, retrieving the information with custom and non-standardized logic.

In the new system, it will be necessary to standardize these dynamics, finding a way to define the additional parameters that will be needed, beyond those of the fields, during action configuration, and to generalize the implementation logic as much as possible, so that if new "groups" of additional parameters (e.g., AWS credentials, O365 credentials, etc.) were to be added, it can be done in the most efficient way possible.

Target Machine Configuration

In the old system, Service Requests were executed exclusively on the customers' Domain Controllers. Specifically, each DC ran an agent, which periodically checked via *polling*[1] if there were any Service Requests in the `PENDING` state with a matching `domain` field.

This part of the architecture will also need to be redesigned, as it is inefficient and highly limiting by design [24]. Therefore, in addition to redesigning these dynamics, it will also be necessary to provide the option to choose, during the configuration of an Action, to execute the related SRs on other types of machines, such as internal Elmec workers, or other types of machines on the customer's network other than the Domain Controller (e.g., Exchange Server). It must also be possible to choose to execute SRs through the old flow, in order to continue supporting both systems, at least initially [15].

2.1.2 Service Request Execution

In the current system, the execution of SRs occurs according to this sequence of main events:

1. The frontend sends the request to create a new SR to the REST service (`SRPWebServiceREST`), with all the necessary data (completed fields, customer, action, etc.) [20];
2. The SR and the relevant field values are created in the database, setting the status to `NEW` [20];
3. An SSIS job, every minute, checks if there are SRs in the `NEW` state, and if any are found, they are processed, moving them to subsequent states (e.g., `WAITING FOR APPROVAL`, `PENDING`, `ASSIGNED`, etc.) depending on the action and customer configuration [16];
4. When the SR is in the `PENDING` state, it means it is ready to be executed by the agent on the customer's DC;
5. Every 10 seconds, the agent on the customer's DC checks if there are SRs in the `PENDING` state for its domain (through calls to the SOAP service (`SRPWebService`)) [21];
6. If any are found, for each one, the corresponding script is downloaded, executed, and the SR status is updated to `COMPLETED` or `ERROR` depending on the execution outcome (through the SOAP service) [21];

[1] *Polling* is a communication technique where a system or device periodically and actively samples the status of an external resource or device to check for readiness or new data.

It has already been discussed in the previous chapter how this flow has numerous criticalities, which make it inefficient and not very scalable. It will therefore be necessary to completely redesign it, taking into consideration the possibility of executing SRs on machines other than DCs, and the possibility of executing them also on machines internal to Elmec, and even virtual machines [15].

Elmec already has an internal *provisioning* system, called *Provision Prime*, which is responsible for executing Ansible playbooks on machines (internal, external, and also virtual) [13], and which could be leveraged for this purpose. It will therefore be necessary to analyze this system, and understand if and how it can be integrated into the new SR execution flow, in order to avoid the polling of agents on the DCs, and to have a generally more reliable and efficient system [15].

A constraint that must be respected, however, is that the action scripts are all in PowerShell, and it is not possible to modify them. This is because they are very numerous, have been developed over the years, have been tested and validated for every single client, and are maintained by different Elmec departments. Therefore, even if the SR execution flow is redesigned, the scripts must be executed as they are. This obviously does not apply to new scripts that will be developed in the future, which can be designed leveraging the new technologies [15].

2.1.3 Master Data Import

Another important component of the system is the one responsible for importing customer master data, necessary for populating some action fields (e.g., UPN domains, user accounts, groups, Organizational Units (OU)[2], databases, etc.). The main difficulty lies in the fact that this information physically resides on the customers' machines (large enterprise clients, with thousands of user accounts and groups, are being discussed), and therefore their retrieval in real-time is not immediate.

In the current system, this import occurs in a manner very similar to the SR scenario, but in reverse. It happens through an agent running on the customers' DCs, which periodically (hourly) (downloads and) executes a series of PowerShell scripts to retrieve the necessary information, and deposits them in a specific system folder (`D:\SRPortal\Anag\`) [21]. These files are then consumed by an SSIS job (one for each type of master data) that imports them into the database. The REST services then expose APIs to retrieve

[2]In Windows Server, an Organizational Unit (OU) is a container within Active Directory (AD) used to logically group objects like user accounts, computer accounts, and security groups, similar to a folder in a file system [1].

this information, which are used by the frontend to populate the action fields [20].

In the new system, it will be necessary to redesign this flow as well, in order to eliminate the dependence on agents on customer DCs. Even in this case, however, the PowerShell scripts cannot be modified [15], and must be executed as they are now. It will therefore be necessary to analyze how to integrate the execution of these scripts into the new flow, perhaps also leveraging Elmec's internal *provisioning* system (*Provision Prime* [13]) in this case, in order to obtain a more traceable, reliable, and efficient system [15].

2.2 New System Architecture

After gathering and defining the requirements for the new *ServiceRequestManager (SRM)* system through various meetings with stakeholders and analysis of the current system (SRP), it has been possible to design a new architecture for the system that allows for efficient and scalable satisfaction of these requirements.

The proposed new architecture for the SRM system, shown in Figure 2.1, is based on a **microservices architecture**, which allows for greater modularity, scalability, and ease of maintenance compared to the distributed-monolithic architecture of the current SRP system [5].

2.2.1 Main Components

The SRM system architecture is composed of the following main components.

Frontend

Two separate web frontends will be developed for end-users and system administrators. These frontends will communicate with the REST services via HTTP/HTTPS APIs.

Due to the business logic being moved to the REST services, the frontends will be lighter and easier to maintain, allowing for greater flexibility in implementing new functionalities and user interface improvements [12]. This also allows for the future integration of a mobile application, which can use the same APIs.

REST Services

REST (Representational State Transfer) is an architectural style for designing web services that leverages HTTP protocols for communication between client and server [7]. The basic idea is that a system's resources (users, orders,

documents, etc.) are exposed through a uniform interface, typically HTTP, using [7]:

- URLs to identify resources (e.g., `/api/catalog/actions/123/`);
- HTTP verbs to operate on resources (`GET`, `POST`, `PUT`, `DELETE`, ...);
- Representations of resources in standard formats (JSON, XML, ...).

These services will be responsible for business logic, user request management, and interaction with databases and other system components.

Authentication and Authorization System

Authentication and authorization will be managed through the internally developed library, PyElmecAuth or GoElmecAuth (depending on the implementation language), which will interface with the corporate identity management system to ensure secure and controlled access to the REST services.

It will therefore be necessary to integrate a granular permission system into SRM to ensure that users can only access the resources and functionalities for which they are authorized, based on the roles defined by the corporate Identity Provider.

Caching System

A caching system will be required to improve system performance and reduce the load on the databases.

In particular, responses from calls to external services will be cached, such as third-party APIs needed to validate some fields, or calls to retrieve customer information, to reduce response times and improve the user experience. The time-to-live (TTL) of the caches must be configurable based on the specific needs of each data type.

Message Broker and Task Queue

To manage asynchronous operations and improve system scalability, a Message Broker (e.g., RabbitMQ, Apache Kafka, Redis, ...) will be used in combination with a Task Queue (e.g., Celery).

This is necessary as the SR lifecycle is very complex and involves many operations that can require long execution times [14], such as sending emails, maintaining HDA[3] tickets, integration with external systems, etc. By using

[3]HDA (Help Desk Advanced) is a web-based ticketing and service management system developed by Zucchetti. It enables organizations to manage tickets, incidents, and support processes through a centralized platform [9].

a messaging and task queue system, these operations can be executed in the background, without blocking the application's main flow [23].

Provisioning System

It will also be necessary to integrate a provisioning system to execute the scripts related to SRs and those for master data import directly within the clients' networks. This system must be secure, reliable, and scalable, in order to be able to manage a large number of provisioning requests simultaneously.

Elmec already has its own provisioning system called *Provision Prime* [13], which will be integrated with the new SRM system to execute scripts efficiently and securely.

Monitoring and Logging System

To ensure the stability and reliability of the system, a monitoring and logging system will be required. This system will allow tracking system performance, identifying any problems, and analyzing logs to resolve bugs. Tools such as Prometheus, Grafana, New Relic, or other similar tools will be used to implement this system.

Both system metrics (CPU, memory, database usage, API response times, etc.) and application logs (errors, warnings, debug information, etc.) will be tracked, in order to always have a complete view of the system status, and to have the possibility of launching automatic alarms in case of problems (e.g., if the number of responses resulting in 500 (Internal Server Error) in the last hour is $\geq 1\%$).

Containerization and Container Orchestration System

To facilitate system deployment, scalability, and management, a containerization system (Docker) will be used in combination with a container orchestration system (Kubernetes).

This will allow isolating the different system components, simplifying the deployment process, and ensuring greater resilience and scalability of the system, through functionalities such as replicas, chronjobs, automatic scaling of pods, and so on [11].

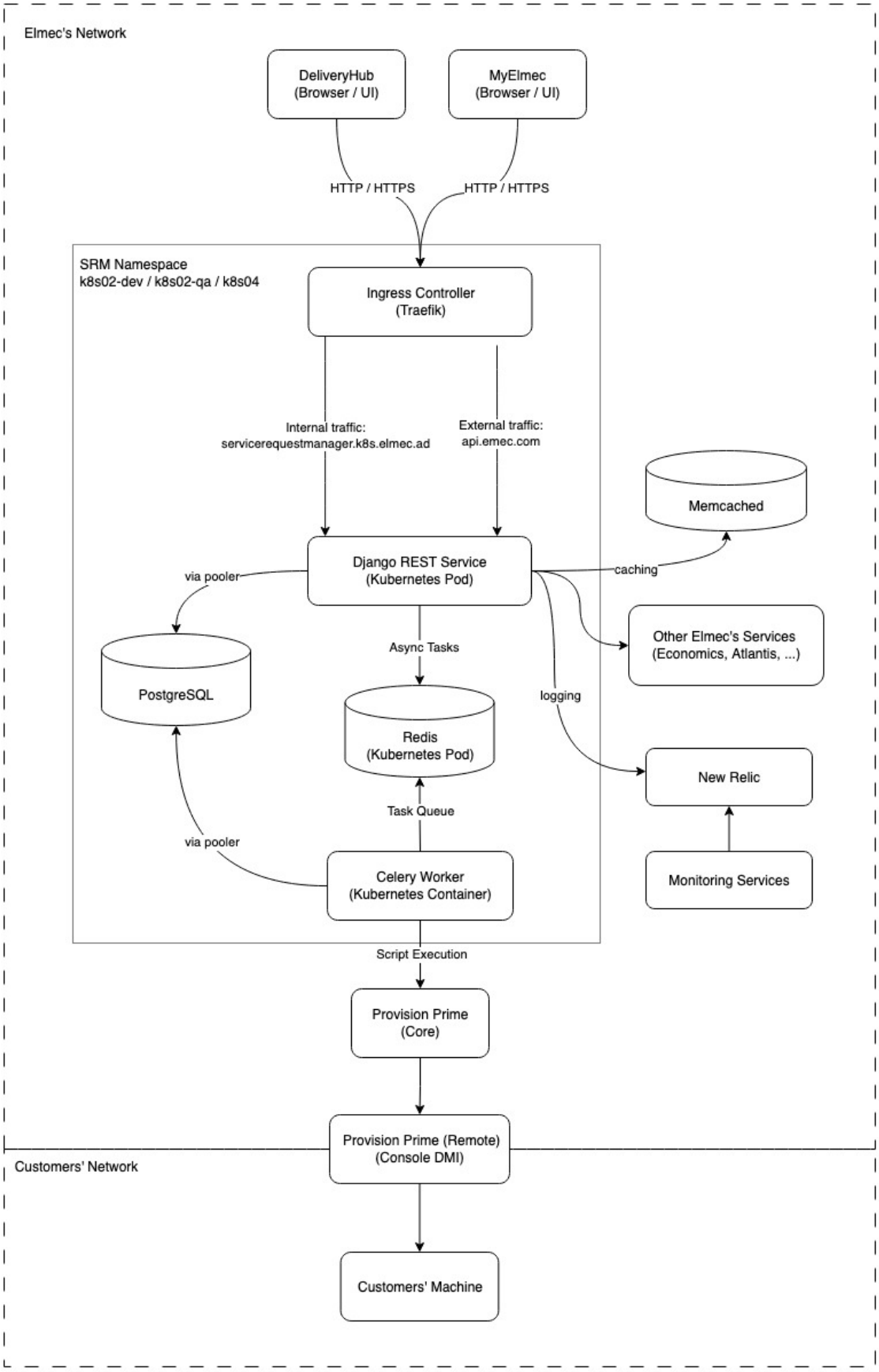

Figure 2.1: SRM System Architecture

Chapter 3

Implementation of a Modular and Scalable Solution

- Overview of technologies to be used (backend, frontend, database)
- Description of tools for automation and containerization
- Reasons behind the choice of said technologies
- Implementation of a modular and scalable solution that can accomodate various client-specific configurations

Chapter 4

Migrations

- Planning and execution of a full migration of databases and other company services / platforms that use SRP;
- Strategies to minimize downtime and ensure data integrity during the migration process
- Strategies to ensure backward compatibility during the transition period
- Strategies to ensure a smooth co-existence of the two systems during the transition period

Chapter 5

Reccomendation system through RAG and Fine-tuned LLMs

- Integration of Retrieval-Augmented Generation (RAG) techniques
- Fine-tuning of Large Language Models (LLMs)
- Implementation of a recommendation system for service requests

Conclusions

- Reflections on the internship experience and acquired skills
- Considerations on the future of the project and potential developments

Bibliography

[1] *Active Directory Organizational Unit (OU)*. URL: https://blog.netwrix.com/2024/02/19/active-directory-organizational-unit-ou/ (cit. on p. 22).

[2] Mashel Albarak and Rami Bahsoon. *Prioritizing Technical Debt in Database Normalization Using Portfolio Theory and Data Quality Metrics*. 2018. arXiv: 1801.06989 [cs.SE]. URL: https://arxiv.org/abs/1801.06989 (cit. on p. 15).

[3] Michael R. Blaha. "Referential Integrity Is Important For Databases". In: 2005. URL: https://api.semanticscholar.org/CorpusID:6831872 (cit. on pp. 14, 15).

[4] *Cryptsetup and LUKS - open-source disk encryption*. URL: https://gitlab.com/cryptsetup/cryptsetup (cit. on p. 10).

[5] Lorenzo De Lauretis. "From Monolithic Architecture to Microservices Architecture". In: *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 2019, pp. 93–96. DOI: 10.1109/ISSREW.2019.00050 (cit. on p. 23).

[6] *DMI Infrastructure*. Internal repository, not publicly accessible. URL: https://git.elmec.com/publicautomation/dmi-infrastructure.git (cit. on pp. 7, 10).

[7] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. University of California, Irvine, 2000 (cit. on pp. 23, 24).

[8] Paul J. Deitel Harvey M. Deitel. *C++. Fondamenti di programmazione*. Apogeo, 2005 (cit. on p. 15).

[9] *HDA - Help Desk Advanced*. URL: https://pat.eu/en/products/helpdeskadvanced/ (cit. on p. 24).

[10] *K3s - Lightweight Kubernetes*. URL: https://docs.k3s.io/ (cit. on p. 10).

[11] *Kubernetes: Production-Grade Container Orchestration*. URL: https://kubernetes.io/docs/concepts/overview/ (cit. on p. 25).

[12] Severi Peltonen, Luca Mezzalira, and Davide Taibi. *Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review*. 2021. arXiv: 2007.00293 [cs.SE]. URL: https://arxiv.org/abs/2007.00293 (cit. on pp. 11, 23).

[13] *Provision Prime*. Internal repository, not publicly accessible. URL: https://git.elmec.com/innovation/development/provisionprime.git (cit. on pp. 22, 23, 25).

[14] Elmec Informatica S.p.A. *Azioni Gestione SRP 4.0*. Internal document, not publicly available., 2016 (cit. on pp. 3, 6, 24).

[15] Elmec Informatica S.p.A. *Requirements definition meeting for the new system*. Internal meeting. Held with the stakeholders and developers of the old system to gather requirements for the new system. Apr. 2025 (cit. on pp. 15, 18–23).

[16] Elmec Informatica S.p.A. *Service Request Portal*. Internal document, not publicly available., 2016 (cit. on pp. 7–9, 21).

[17] *SQL Server Integration Services*. URL: https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services (cit. on p. 14).

[18] *SRP Internal Worker Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srpinternalworker_windowsservice.git (cit. on p. 9).

[19] *SRP Web Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WebService.git (cit. on p. 9).

[20] *SRP Web Service REST*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srp_webservicerest.git (cit. on pp. 8, 18, 21, 23).

[21] *SRP Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WindowsService.git (cit. on pp. 7, 9, 15, 21, 22).

[22] *Stored Procedures (Database Engine)*. URL: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine (cit. on p. 14).

[23] Jan Tretmans and Louis Verhaard. "A Queue Model Relating Synchronous and Asynchronous Communication". In: *Protocol Specification, Testing and Verification, XII*. Ed. by R.J. LINN and M.Ü. UYAR. IFIP Transactions C: Communication Systems. Amsterdam: Elsevier, 1992, pp. 131–145. ISBN: 978-0-444-89874-6. DOI: https://doi.org/10.1016/B978-0-444-89874-6.50015-5. URL: https://www.sciencedirect.com/science/article/pii/B9780444898746500155 (cit. on p. 25).

[24] Brecht Van de Vyvere, Pieter Colpaert, and Ruben Verborgh. "Comparing a Polling and Push-Based Approach for Live Open Data Interfaces". In: *Web Engineering*. Ed. by Maria Bielikova, Tommi Mikkonen, and Cesare Pautasso. Cham: Springer International Publishing, 2020, pp. 87–101. ISBN: 978-3-030-50578-3 (cit. on p. 21).

[25] *What is referential integrity and what does it look like in practice?* URL: https://www.y42.com/blog/referential-integrity (cit. on p. 14).